



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
  
FUNCTION(Symbol) replicate  
    x = (function(x,y){return x+y;});  
    class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 11η **ΠΑΡΑΓΩΓΗ ΕΝΔΙΑΜΕΣΟΥ ΚΩΔΙΚΑ – III (τελευταία)**

Περιεχόμενα

- *Εκφράσεις*
- Εντολή διακλάδωσης
- Ανακυκλώσεις
- Ειδικές εντολές

Εκφράσεις (1/4)

- Αριθμητικών τελεστών
- Συσχετιστικών τελεστών
- Λογικών τελεστών

Εκφράσεις (2/4)

Αριθμητικών τελεστών

```
arithop → + { $arithop = add; }
arithop → - { $arithop = sub; }
arithop → * { $arithop = mul; }
arithop → / { $arithop = div; }
arithop → % { $arithop = mod; }
```

```
expr → expr1 arithop expr2
{
    $expr = newexpr(arithexpr_e);
    $expr->sym = newtemp();
    emit($arithop, $expr1, $expr2, $expr);
}
```

Εδώ επίσης δεν δείχνουμε και την εφαρμογή της τακτικής επαναχρησιμοποίησης κρυφών μεταβλητών, ωστόσο μπορεί να εφαρμοστεί κανονικά.

$a = (x+y) - (z*w)$	1: ADD x y _t1
	2: MUL z w _t2
	3: SUB _t1 _t2 _t3
	4: ASSIGN _t3 a

Η παραπάνω είναι η πολύ απλή περίπτωση υλοποίησης. Υπάρχουν και οι εξής περιπτώσεις που πρέπει να ελεγχθούν:

- ❑ Εάν κάποιο expression είναι ήδη γνωστού τύπου που δεν επιτρέπεται να συμμετέχει σε αριθμητική έκφραση, όπως: *programfunc_e*, *libraryfunc_e*, *boolexpr_e*, *newtable_e*, *constbool_e*, *conststring_e* και *nil_e*.
- ❑ Εάν έχουμε δύο expressions τύπου *constnum_e*, τότε υπολογίζουμε το αποτέλεσμα και το *\$expr* γίνεται και αυτό τύπου *constnum_e*.

Εκφράσεις (3/4)

Συσχετιστικών τελεστών (ολική αποτίμηση)

$relop \rightarrow > \quad \{ \$relop = \text{if_greater}; \}$
 $relop \rightarrow >= \quad \{ \$relop = \text{if_greatereq}; \}$
 $relop \rightarrow < \quad \{ \$relop = \text{if_less}; \}$
 $relop \rightarrow <= \quad \{ \$relop = \text{if_lesseq}; \}$
 $relop \rightarrow == \quad \{ \$relop = \text{if_eq}; \}$
 $relop \rightarrow != \quad \{ \$relop = \text{if_noteq}; \}$

$expr \rightarrow expr_1 relop expr_2$
 $\{$

$\$expr = \text{newexpr}(\text{boolexpr}_e);$
 $\$expr \rightarrow sym = \text{newtemp}();$

$\text{emit}(\$relop, \$expr_1, \$expr_2, \text{nextquad}()+3);$
 $\text{emit}(\text{assign}, \text{newexpr_constbool}(0), \$expr);$
 $\text{emit}(\text{jump}, \text{nextquad}()+2);$
 $\text{emit}(\text{assign}, \text{newexpr_constbool}(1), \$expr);$

$\}$

$a = x >= y;$	1: IF_GREATEREQ x y 4 2: ASSIGN _t1 FALSE 3: JUMP 5 4: ASSIGN _t1 TRUE 5: ASSIGN _t1 a
---------------	--

$b = f() >= g();$	1: CALL f 2: GETRETVAL _t1 3: CALL g 4: GETRETVAL _t2 5: IF_GREATEREQ _t1 _t2 8 6: ASSIGN _t3 FALSE 7: JUMP 9 8: ASSIGN _t3 TRUE 9: ASSIGN _t3 b
-------------------	--

Και σε αυτή την περίπτωση παραλείπονται οι στατικοί έλεγχοι συμφωνίας τύπων αλλά και υπολογισμού τιμής της λογικής έκφρασης από σταθερά ορίσματα (στην περίπτωση αυτή προφανώς δεν παράγεται κώδικας αφού ο υπολογισμός γίνεται από τον compiler).

Εκφράσεις (4/4)

Λογικών τελεστών (ολική αποτίμηση)

$boolop \rightarrow \&\& \{ \$boolop = \text{and}; \}$

$boolop \rightarrow || \{ \$boolop = \text{or}; \}$

$a = x || y \&\& z;$

1: AND $y \ z \ t1$

2: OR $x \ t1 \ t2$

3: ASSIGN $t2 \ a$

$expr \rightarrow expr_1 boolop expr_2$

{

$\$expr = newexpr(boolexpr_e);$

$\$expr \rightarrow sym = newtemp();$

$emit(\$boolop, \$expr_1, \$expr_2, \$expr);$

}

Η μέθοδος μερικής αποτίμησης είναι ιδιαίτερα πονηρή, εσείς ακολουθείτε πιστά τη μέθοδο (φροντιστήριο)

Και πάλι ισχύουν όσα και στις προηγούμενες περιπτώσεις εκφράσεων, μόνο που όταν έχουμε σταθερές τιμές η συναρτήσεις ή δυναμική κατασκευή πίνακα, γνωρίζουμε ότι είναι μετατρέψιμες σε τιμή boolean και τις θεωρούμε constants.

function f(){
a = (t=[]) || f && print;

1: NEWTABLE t

2: ASSIGN TRUE a

??

or

true

true and true

true

Θα μπορούσαμε να τροποποιήσουμε την υλοποίηση του assignment ώστε εάν γνωρίζουμε τον τύπο του r-value να τον διοχετεύσουμε και στο assign expression (το οποίο τώρα φέρει τύπο απλώς *assignexpr_e*). Έτσι θα μπορούσαμε να γνωρίζουμε ότι το (t=[]) είναι δυναμικός πίνακας που είναι true.

Περιεχόμενα

- Εκφράσεις
- *Εντολή διακλάδωσης*
- Ανακυκλώσεις
- Ειδικές εντολές

Εντολή διακλάδωσης (1/4)

if (expr) stmt (1/2)

Ενδιάμεσος κώδικας που παράγεται
λόγω του expr

IF_EQ \$expr TRUE
JUMP

Το πρώτο quad του stmt

Ενδιάμεσος κώδικας που παράγεται
λόγω του *stmt*

Το πρώτο quad ακριβώς μετά το *stmt*

+2

•Εναλλακτικά μπορούμε να έχουμε IF_NE με jump στο πρώτο quad μετά το stmt που σημαίνει ότι δεν απαιτείται το JUMP.

•Πρέπει με κάποιο τρόπο να μπορέσουμε να προκαλέσουμε την παραγωγή του ενδιάμεσου κώδικα των δύο αυτών εντολών ανάμεσα στο *expr* και στο *stmt*.

•Επίσης, ο μόνος τρόπος να γνωρίζουμε ακριβώς την θέση των τελικών quads και για τις δύο εντολές είναι αφού έχει παραχθεί ο κώδικας για το *stmt*.

if (x+y >= z-w)
f(x,y,z);

```
1: ADD x y _t1
2: SUB z w _t2
3: IF_GREATEREQ _t1 _t2 6
4: ASSIGN FALSE _t3
5: JUMP 7
6: ASSIGN TRUE _t3
7: IF_EQ TRUE _t3 9
8: JUMP 14
```

```
9: PARAM x
10: PARAM y
11: PARAM z
12: CALL f
13: GETRETVAL _t4
```

14:

go if

skip if



Εντολή διακλάδωσης (2/4)

if (*expr*) *stmt* (2/2)

```
ifprefix → if ( expr )
{
    emit(
        if_eq, $expr,
        newexpr_constbool(1),
        nextquad() + 2
    );
    $ifprefix = nextquad();
    emit(jump, NULL, NULL, 0);
}
if → ifprefix stmt
{
    patchlabel($ifprefix, nextquad());
}
```

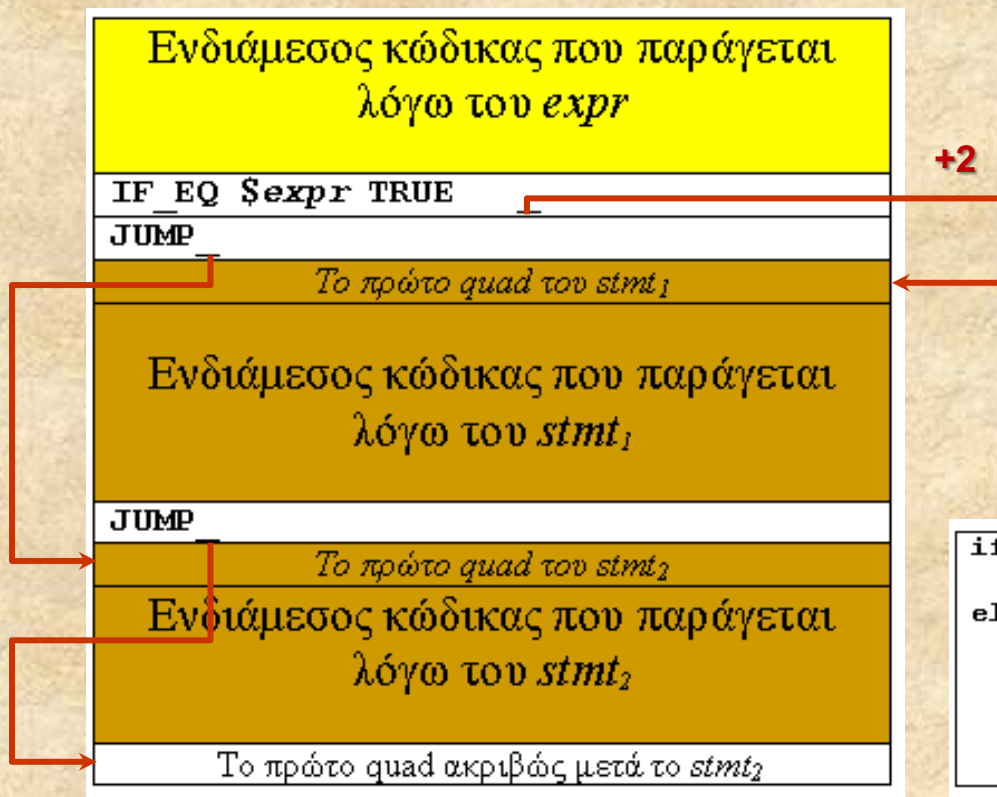
```
void patchlabel (unsigned quadNo, unsigned label) {
    assert(quadNo < currQuad && !quads[quadNo].label);
    quads[quadNo].label = label;
}
expr* newexpr_constbool (unsigned int b) {
    expr* e = newexpr(constbool_e);
    e->boolConst = !!b;
    return e;
}
unsigned nextquad (void) { return currQuad; }
```

•Βλέπουμε ότι λύνουμε εύκολα το πρόβλημα παραγωγής κώδικας ανάμεσα στο *expr* και *stmt* με την εισαγωγή ενός επιπλέον κανόνα ο οποίος παράγει τις δύο αναγκαίες εντολές *if_eq* και *jump*.

•Ουσιαστικά ο σημασιολογικός κανόνας του *ifprefix* καλείται μετά την παραγωγή του κώδικα για το *expr* αλλά πάντα πριν την παραγωγή του κώδικα για το *stmt* (καθώς το *ifprefix* γίνεται *reduced* πριν καν γίνει *parsed* οτιδήποτε παράγεται από το *stmt*).

Εντολή διακλάδωσης (3/4)

if (*expr*) *stmt* else *stmt* (1/2)



+2

•Και πάλι έχουμε παρόμοιο πρόβλημα με πριν καθώς είναι αναγκαίο να εισάγουμε ένα JUMP ανάμεσα στον κώδικα των δύο statements.

•Και πάλι εφαρμόζουμε την ίδια λύση με την εισαγωγή ενός γραμματικού κανόνα που γίνεται πάντα reduced μετά το 1ο *stmt* αλλά πάντα πριν το 2ο *stmt*.

<code>if (a b)</code>	<code>1: OR a b t1</code>
<code> x = y;</code>	<code>2: IF_EQ TRUE _t1 4</code>
<code>else</code>	<code>3: JUMP 6</code>
<code> y = z;</code>	<code>4: ASSIGN y x</code>
	<code>5: JUMP 7</code>
	<code>6: ASSIGN z_y</code>
	<code>7:</code>

Εντολή διακλάδωσης (4/4)

if (*expr*) *stmt* else *stmt* (2/2)

```
elseprefix → else
{
    $elseprefix = nextquad();
    emit(jump, NULL, NULL, 0);
}
if → ifprefix stmt1 elseprefix stmt2
{
    patchlabel($ifprefix, $elseprefix + 1);
    patchlabel($elseprefix, nextquad());
}
```

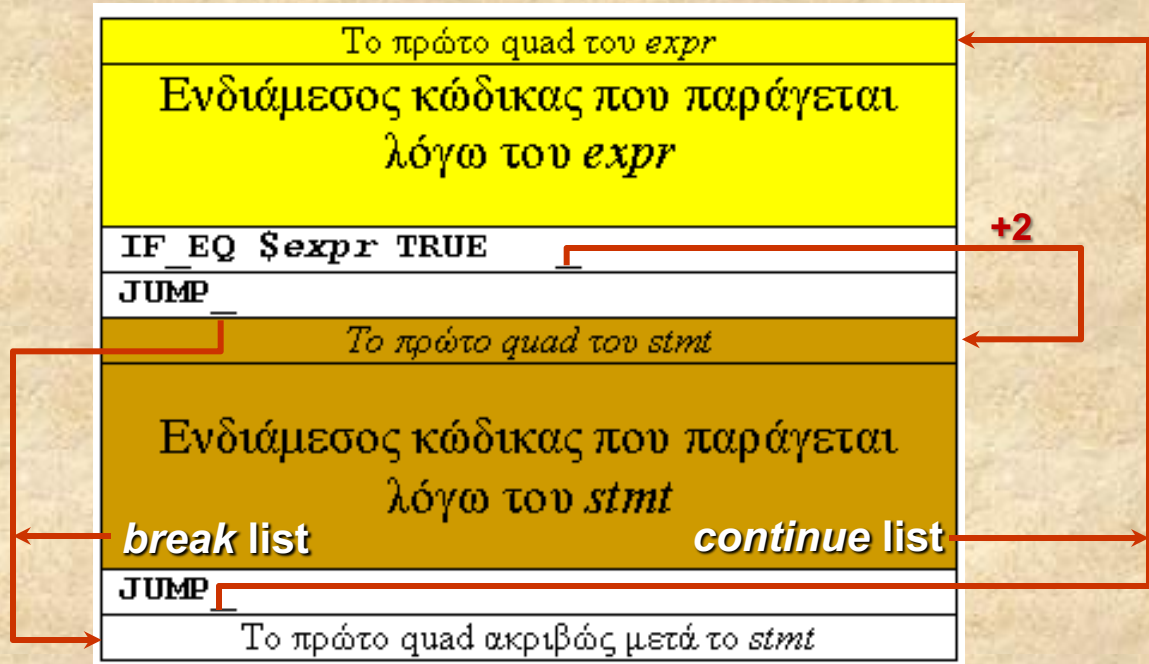
- Η τιμή του *elseprefix* είναι ο αριθμός του quad ακριβώς πριν το 1ο quad του *stmt*₂, και αντιστοιχεί σε ένα εμβόλιμο JUMP που ακολουθεί την εκτέλεση του *stmt*₁.
- Η τιμή του *ifprefix* αντιστοιχεί στο JUMP στην περίπτωση που το if expression είναι false, άρα θα πρέπει να οδηγεί στο 1ο quad του *stmt*₂. Όμως από τα προηγούμενα, ο αριθμός αυτού του quad είναι ακριβώς *\$elseprefix*+1, άρα αρκεί να κάνουμε patch το target label του quad *\$ifprefix* με την τιμή αυτή.
- Τώρα, καθώς το quad στη θέση *\$elseprefix* αντιπροσωπεύει το JUMP όταν περατωθεί η εκτέλεση του *stmt*₁ θα πρέπει να γίνει patched με τον αριθμό του 1ου quad αμέσως μετά το *stmt*₂. Όμως τη στιγμή που εκτελείται ο σημασιολογικός κανόνας, η τιμή αυτή είναι ακριβώς το *nextquad()*.

Περιεχόμενα

- Εκφράσεις
- Εντολή διακλάδωσης
- Ανακυκλώσεις
- Ειδικές εντολές

Ανακυκλώσεις (1/5)

while (expr) stmt (1/2)



```

1: SUB x y _t1
2: SUB z w _t2
3: IF_GREATER _t1 _t2 6
4: ASSIGN FALSE _t3
5: JUMP 7
6: ASSIGN TRUE _t3

```

```

7: IF_EQ TRUE _t3 9
8: JUMP 16

```

```

9: IF_EQ TRUE b 11
10: JUMP 13
11: JUMP 16
12: JUMP 15
13: SUB x z _t4
14: ASSIGN _t4 x

```

```

15: JUMP 1
16:

```

```

while (x-y > z-w)
    if (b)
        break;
    else
        x = x-z;

```

- Εδώ οι ανάγκες είναι ίδιες με το απλό if ως προς την πρώτη φορά εκτέλεσης. Όμως επειδή πρόκειται για ανακύκλωση (δηλ. επαναλαμβανόμενο if) πρέπει στο τέλος να έχουμε πάντα ένα jump στην αρχή, στο 1ο quad υπολογισμού του *expr*.
- Επίσης, καθώς μέσα στο *stmt* μπορεί να υπάρχουν εντολές `break` ή `continue`, θα δούμε ότι αυτές προκαλούν την παραγωγή μη συμπληρωμένων jumps (unfinished) τα οποία κρατούνται σε μία λίστα ώστε να συμπληρωθούν από τον σημασιολογικό κανόνα του `while` statement. Προφανώς τα `break` πρέπει να γίνουν patched στο quad που ακριβώς έπεται του *stmt*, ενώ τα `continue` στο 1ο quad υπολογισμού του *while* expression.

Ανακυκλώσεις (2/5)

while (*expr*) *stmt* (2/2)

```
whilestart → while
{
    $whilestart = nextquad();
}
whilecond → (expr)
{
    emit(
        if_eq,
        $expr,
        newexpr_constbool(1),
        nextquad() + 2
    );
    $whilecond = nextquad();
    emit(jump, NULL, NULL, 0);
}
while → whilestart whilecond stmt
{
    emit(jump, NULL, NULL, $whilestart);
    patchlabel($whilecond, nextquad());
    patchlist($stmt.breaklist, nextquad());
    patchlist($stmt.contlist, $whilestart);
}
```

• Στο *whilestart* κρατάμε το 1ο quad του κώδικα υπολογισμού της συνθήκης της εντολής *while*.

• Στο *whilecond* παράγουμε τον κώδικα που ελέγχει την τιμή του *while expression* και κάνει αντίστοιχα *jump* ή στην αρχή του *stmt* ή στο τέλος του. Καθώς ο αριθμός του quad αμέσως μετά το τέλος του *stmt* δεν είναι γνωστός, κρατάμε τον αριθμό του *jump quad* ως γνώρισμα του *whilecond* για να το κάνουμε *patch* στον βασικό κανόνα.

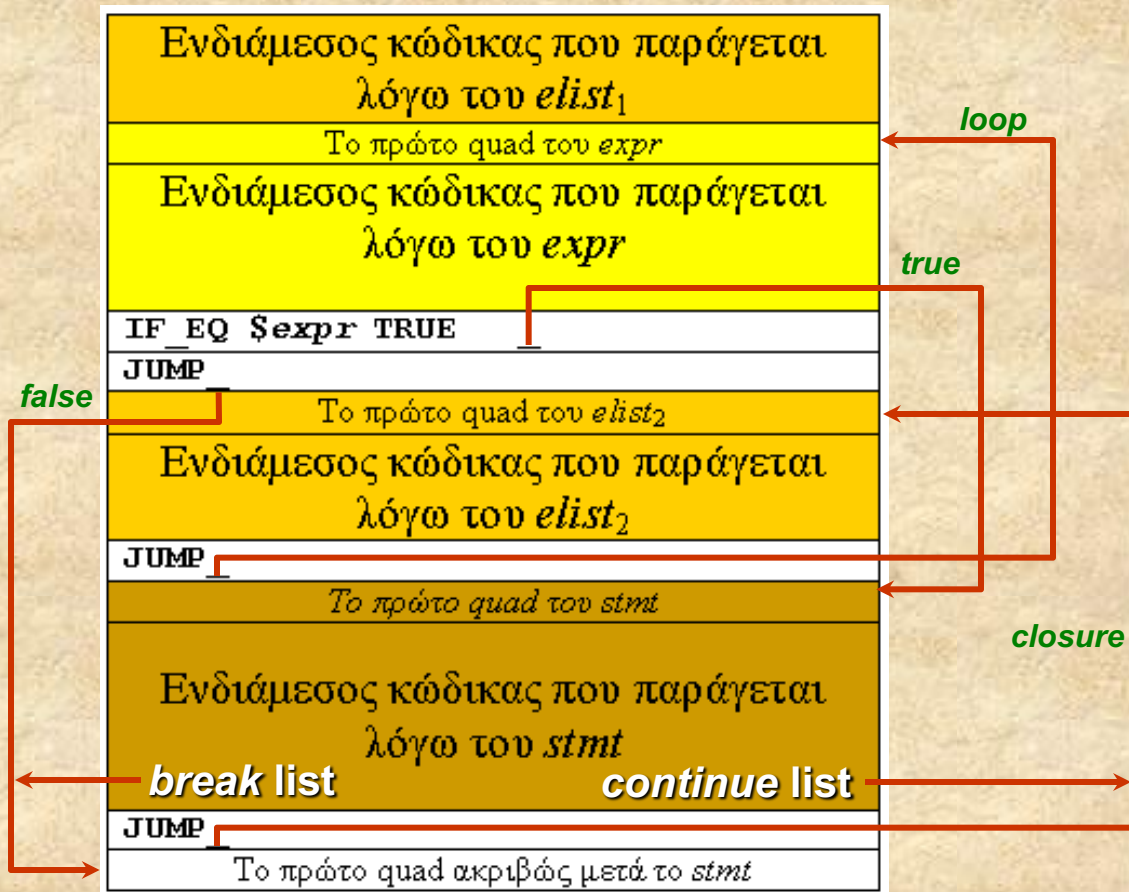
• Στο τέλος του *while stmt* πάντα υπάρχει *JUMP* στην αρχή του, με τιμή αρχικού quad να είναι αποθηκευμένη στο γνώρισμα του *whilestart*.

Ανακυκλώσεις (3/5)

for (elist; expr; elist) stmt (1/3)

Κάθε for loop μπορεί να γραφεί ως ένα ισοδύναμο while loop.

for (elist₁; expr; elist₂) stmt
elist₁ while (expr) { stmt elist₂ }



•Οι συνδέσεις είναι υποχρεωτικές καθώς ο κώδικας παράγεται πάντα με τη σειρά αναγνώρισης των γραμματικών συμβόλων, που είναι και η σειρά εμφάνισης στο RHS, δηλ. *elist₁*, *expr*, *elist₂* και έπειτα *stmt*.

•Άρα θα πρέπει να φροντίσουμε και πάλι να έχουμε εμβόλιμα τα JUMP quads, ώστε να υποστηρίζεται η σημασιολογία του for loop, μέσω ειδικών γραμματικών συμβόλων και παραγωγών.

Ανακυκλώσεις (4/5)

for (*elist*; *expr*; *elist*) *stmt* (2/3)

$N \rightarrow \{ \$N = \text{nextquad}(); \text{emit}(\text{jump}, \text{NULL}, \text{NULL}, 0); \}$

$M \rightarrow \{ \$M = \text{nextquad}(); \}$

forprefix $\rightarrow$ for (*elist* ; M *expr* ;

```
{  
    $forprefix.test  = $M;  
    $forprefix.enter = nextquad();  
    emit(if_eq, $expr, newexpr_constbool(1), 0);  
}
```

for $\rightarrow$ *forprefix* N_1 *elist*) N_2 *stmt* N_3

```
{  
    patchlabel($forprefix.enter, $N_2+1);  $\leftarrow$  true jump  
    patchlabel($N_1, nextquad());  $\leftarrow$  false jump  
    patchlabel($N_2, $forprefix.test);  $\leftarrow$  loop jump  
    patchlabel($N_3, $N_1+1);  $\leftarrow$  closure jump  
  
    patchlist($stmt.breaklist, nextquad());  
    patchlist($stmt.contlist, $N_1+1);  
}
```

•Ο κανόνας N εισάγεται για να παράγει ένα unfinished JUMP ενώ κρατάει σαν γνώρισμα το quad number αυτού του JUMP. Καθώς είναι γραμματικά της μορφής $N \rightarrow \epsilon$, κάνει πάντα reduce όταν τα γραμματικά σύμβολα στα αριστερά του είναι στη στοίβα.

•Καθώς χρειαζόμαστε τρία (3) τέτοια εμβόλιμα JUMPs, βάζουμε στα τρία κατάλληλα σημεία του βασικού κανόνα το γραμματικό σύμβολο N .

•Επίσης, πρέπει να κρατήσουμε και τον αριθμό του quad όπου γίνεται test το for expression, καθώς πρέπει να κάνουμε patch το jump ακριβώς στο πρώτο quad του *stmt*.

Ανακυκλώσεις (5/5)

for (elist; expr; elist) stmt (3/3)

<i>elist₁</i>	<i>expr</i>	<i>elist₂</i>
for (i = 0;	i < N;	++i)
print("*");		
<i>stmt</i>		

<i>elist₁</i>	1: ASSIGN 0 i	
<i>expr</i>	2: IF_LESS i N 5	
	3: ASSIGN FALSE _t1	
	4: JUMP 6	
	5: ASSIGN TRUE _t1	
	6: IF EQ TRUE t1 10	<i>true jump</i>
	7: JUMP 14	<i>false jump</i>
<i>elist₂</i>	8: ADD i 1 i	
	9: JUMP 2	<i>loop jump</i>
<i>stmt</i>	10: PARAM "*"	
	11: CALL "print"	
	12: GETRETVAL _t2	
	13: JUMP 8	<i>closure jump</i>
	14:	

Περιεχόμενα

- Εκφράσεις
- Εντολή διακλάδωσης
- Ανακυκλώσεις
- *Ειδικές εντολές*



Ειδικές εντολές (1/6)

- **break** και **continue**
- **return** [*expr*]

Ειδικές εντολές (2/6)

- Το **break** πρέπει να προκαλεί αλλαγή ροής ελέγχου ώστε να έχουμε άμεση έξοδο από το εκάστοτε loop.
- Το **continue** πρέπει να προκαλεί άμεση κλείσιμο της παρούσας ανακύκλωσης και έναρξη της επόμενης στο εκάστοτε loop.
- Καθώς και τα δύο χρησιμοποιούνται μόνο μέσα σε loop θα πρέπει να μπορείτε να ελέγχετε εάν βρίσκονται μέσα σε κώδικα κάποιου loop.
- Καθώς και τα δύο κάνουν jumps σε σημεία (quads) του ενδιαμέσου κώδικα που δεν είναι γνωστά μέσα στον ίδιο τον γραμματικό τους κανόνα, γίνονται emit ως unfinished jumps.



Ειδικές εντολές (3/6)

```
loopstart → ε          { ++loopcounter; }
loopend → ε             { --loopcounter; }
loopstmt → loopstart stmt loopend { $ loopstmt = $stmt; }
```

```
whilestmt → while ( expr ) loopstmt
forstmt   → for ( elist; expr; elist ) loopstmt
```

```
funcblockstart → ε      { push_loopcounter(); }
funcblockend → ε        { pop_loopcounter(); }
```

```
funcdef → function [id] (idlist) funcblockstart block funcblockend
```

```
struct lc_stack_t {
    struct lc_stack_t* next;
    unsigned           counter;
} *lcs_top = 0, *lcs_bottom = 0;
```

```
#define loopcounter \
    (lcs_top->counter)
```

```
extern void push_loopcounter (void);
extern void pop_loopcounter  (void);
```

...

On compilation start:

push_loopcounter(); ← 0 default value

Κώδικας	loopcounter stack και current value
x = f();	[0]
while (x) {	[1]
function g() {	[1] [0], <i>push, reset</i>
break;	[1] [0], Άρα compile error ("not in a loop")
while(true) {	[1] [1]
continue;	[1] [1], Νόμιμο, μέσα σε loop
}	[1] [0]
}	[1], <i>pop, restore</i>
break;	[1], Νόμιμο, μέσα σε loop
}	[0]

• Δείχνουμε μόνο το τμήμα των σημασιολογικών κανόνων για τη διαχείριση του *loopcounter*.

• Προφανώς η λογική που παρουσιάζεται πρέπει να ενσωματωθεί στους κανόνες που έχουμε ήδη παρουσιάσει για τις ανακυκλώσεις.

Ειδικές εντολές (4/6)

```
break → break ;
{
    make_stmt(&$break);
    $break.breaklist = newlist(nextquad()); emit(jump, NULL, NULL, 0);
}
continue → continue ;
{
    make_stmt(&$continue);
    $continue.contlist = newlist(nextquad()); emit(jump, NULL, NULL, 0);
}
stmts → stmt { $stmts = $stmt; }
stmts → stmts1 stmt
{
    $stmts.breaklist = mergelist($stmts1.breaklist, $stmt.breaklist);
    $stmts.contlist = mergelist($stmts1.contlist, $stmt.contlist);
}
```

• Για την υποστήριξη των `break` και `continue`, εισάγουμε δύο γνωρίσματα στο γραμματικό σύμβολο `stmt`: (α) την λίστα `breaklist`, που περιέχει τους αριθμούς όλων των quads που καθένα αντιστοιχεί σε unfinished jump λόγω κάποιου `break` που περιέχεται στο `stmt`, και (β) το `contlist` αντίστοιχα, με τους αριθμούς όλων των quads που καθένα αντιστοιχεί σε unfinished jump λόγω κάποιου `continue` που περιέχεται στο `stmt`.

• Για τον διπλανό πηγαίο κώδικα, και για το `stmt` που αντιστοιχεί σε όλο το block του loop, σκιαγραφούμε τις εντολές που θα δώσουν quad indices που τελικά θα περιλαμβάνονται στις δύο αυτές λίστες.

```
for (i=0; i<N; ++i)
{
    if (f(x)[i] > y)
        break;
    else {
        z = g(y);
        continue;
    }
    h();
}
```


Ειδικές εντολές (5/6)

Παράδειγμα για παραγωγή και συμπλήρωση των jumps για break και continue

while (a) {	1: IF_EQ TRUE a 3	
	2: JUMP 15	// go exit
a = f(a);	3: PARAM a	
	4: CALL f	
	5: GETRETVAL _t1	
	6: ASSIGN _t1 a	
if (a)	7: IF_EQ a TRUE 9	// go if
break;	8: JUMP 11	// go else
else	9: JUMP 15	// break
continue;	10: JUMP 12	// skip else
	11: JUMP 1	// continue
continue;	12: JUMP 1	// continue
break;	13: JUMP 15	// break
}	14: JUMP 1	// go loop
	15:	

Ειδικές εντολές (6/6)

- Το **return** πρέπει να επιτρέπεται μόνο μέσα σε συναρτήσεις, άρα απαιτείται και κάποιος *infunction* μετρητής που τον διαχειρίζεστε ώστε να είναι >0 εάν το παρόν `return stmt` είναι μέσα στο block κάποιας συνάρτησης.

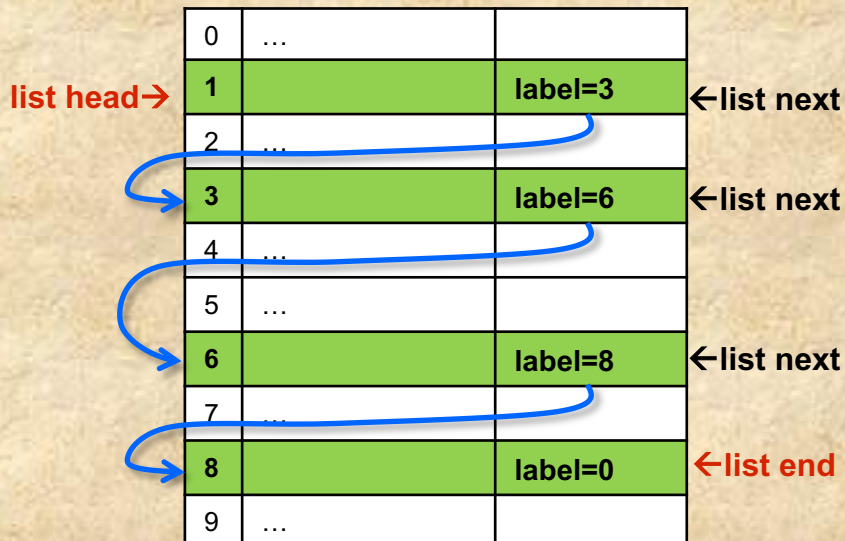
```
returnstmt → return ε ;  
            { emit(return, null); }  
returnstmt → return expr;  
            { emit(return, $expr); }
```

<pre>function f() { return; return local x; return nil; }</pre>	<pre>1: FUNCSTART f 2: RETURN 3: RETURN x 3: RETURN nil 4: FUNCEND f</pre>
---	---



Quad lists

```
struct stmt_t {  
    int breakList, contList;  
};  
  
void make_stmt (stmt_t* s)  
{ s->breakList = s->contList = 0; }  
  
int newlist (int i)  
{ quads[i].label = 0; return i; }
```



```
int mergelist (int l1, int l2) {  
    if (!l1)  
        return l2;  
    else  
        if (!l2)  
            return l1;  
    else {  
        int i = l1;  
        while (quads[i].label)  
            i = quads[i].label;  
        quads[i].label = l2;  
        return l1;  
    }  
}  
  
void patchlist (int list, int label) {  
    while (list) {  
        int next = quads[list].label;  
        quads[list].label = label;  
        list = next;  
    }  
}
```